System Programming

Programming Assignment #1

411221420 林彥宏

A. Brief description of the assignment problem

The goal is to write an SIC assembler in the C programming language. The assembler must read an SIC assembly program source file, translate the SIC statements into their machine code equivalents, and generate an object program file (.obj). The output object program must strictly follow the standard SIC object code format, including the Header record, Text records, and End record.

B. Highlights on how you write the program

Using a classic Two-Pass Assembler architecture:

1. Pass 1 (Symbol Definition):

i. Reads the source file line by line using fgets.
ii. Parses each line into label, opcode, and operand fields.
iii. Maintains a Location Counter (locctr) to calculate the address of each instruction.
iv. Builds a Symbol Table (symtab) using a Linked List structure to dynamically store labels and their corresponding memory addresses.

2. Pass 2 (Object Code Generation):

i. Resets the file pointer and reads the source code again.
ii. Looks up mnemonics in a hardcoded 2D array (optab) to find the machine hex code.
iii. Searches the Symbol Table to find the target addresses for operands.
iv. Handles Indexed Addressing (checking for ,X) by adding 0x8000 to the target address.
v. Uses a buffer (obj_line) to construct Text records. When the buffer reaches its capacity limit, it automatically writes the record to the .obj file and initializes a new one.

C. Program listing

```
#include <stdio.h>
#include <stdlib.h>
```

```
#include <string.h>

#define MAXOP 27

char name[16];
FILE *f;
FILE *fobj;

char line[80];
char label[7];
char op[7];
char operand[10];
char new_operand[7];
int indexed = 0;

char prog_name[7];
int start_addr = 0;
int prog_len = 0;
char obj_line[70];
char obj_code[7];
int locctr = 0;
int textpos = 0;

const char a_start[] = "START";
const char a_end[] = "END";
const char a_byte[] = "BYTE";
const char a_word[] = "WORD";
const char a_resb[] = "RESB";
const char a_resw[] = "RESW";

const char optab[26][2][6] = {{"ADD", "18"}, {"AND", "40"}, {"COMP", "28"}, {"DIV",
"24"}, {"J", "3C"}, {"JEQ", "30"}, {"JGT", "34"}, {"JLT", "38"}, {"JSUB", "48"}, {"LDA",
"00"}, {"LDCH", "50"}, {"LDL", "08"}, {"LDX", "04"}, {"MUL", "20"}, {"OR", "44"},
{"RD", "D8"}, {"RSUB", "4C"}, {"STA", "0C"}, {"STCH", "54"}, {"STL", "14"}, {"STSW",
"E8"}, {"STX", "10"}, {"SUB", "1C"}, {"TD", "E0"}, {"TIX", "2C"}, {"WD", "DC"}};
```

```
const char hex_c[16] = {'0', '1', '2', '3', '4', '5', '6', '7', '8', '9', 'A', 'B', 'C', 'D', 'E', 'F'};

typedef struct sym_node {
	char l[7];
	int v;
	struct sym_node * next;
} symNODE;

symNODE *symtab = NULL;

symNODE* insert(char *s, int r) {
	symNODE *t = NULL;
	t = (struct sym_node *)malloc(sizeof(struct sym_node));
	if(t) {
		strcpy(t->l, s);
		t->v = r;
		t->next = symtab;
	}
	return t;
}

symNODE* search(symNODE *t, char *s) {
	if(t) {
		if (strcmp(s, t->l) == 0) return t;
		else return search(t->next, s);
	} else {
		return NULL;
	}
}

symNODE* new_search( char *s) {
	strcpy( new_operand, s );
	for (int i=strlen(new_operand); i<6; i++)
		new_operand[i] = ' ';
	new_operand[6] = '\0';
```

```
        return search(symtab, new_operand);
}

char* lookup (char *s) {
        int i = 0;
        int nf = 1;
        while ((i < MAXOP) && (nf)) {
                if (strcmp(s, optab[i][0]) == 0) nf=0;
                else i++;
        }
        if (i >= MAXOP) return NULL;
        else return (char*)optab[i][1];
}

int operand_len () {
        int i, l;
        l = strlen(operand);
        // for (i=0; i<l; i++) printf("operand[%d] = [%c].\n", i, operand[i]);
        if (operand[0] == 'C') l -= 3;
        else if (operand[0] == 'X') l = (l-3) / 2;
        // printf("find_length([%s] = %d.\n", operand, l);
        return l;
}

int readline(){
        int i, j, l, x;
        while (fgets(line, 80, f) != NULL) {
                l = strlen(line);
                if ((l > 0) && (line[0] != '.') && (line[0] != '\n') && (line[0] != '\r')) {
                        for (i = 0; i < 6; i++)
                                label[i] = line[i];
                        label[i] = '\0';
                        while(line[i]==' ') i++;
                        j = 0;
                        while((line[i]!=' ')&&(line[i]!='\0')&&(line[i]!='\n')&&(line[i]!='\r')&&(i<l)){
```

```
                op[j] = line[i];
                i++; j++;
            }
            op[j] = '\0';
            while(line[i]==' ') i++;
            j = 0;
            while((line[i]!=' ')&&(line[i]!='\0')&&(line[i]!='\n')&&(line[i]!='\r')&&(i<l)){
                operand[j] = line[i];
                i++; j++;
            }
            operand[j] = '\0';
            indexed = 0; x = strlen(operand);
            if((x>2) && (operand[x-2]==',') && (operand[x-1]=='X')) {
                operand[x-2] = '\0';
                indexed = 1;
            }
            return 1;
        }
    }
    return 0;
}

void pass1 () {
    locctr = 0;
    while(readline()){
        if(strcmp(op, a_start) == 0){
            sscanf(operand, "%x", &start_addr);
            locctr = start_addr;
            strcpy(prog_name, label);
            for(int i = strlen(prog_name); i<6; i++)
                prog_name[i] = ' ';
            prog_name[6] = '\0';
            continue;
        }
        if(strcmp(op, a_end) == 0) break;
```

```
            if(label[0] != ' ' && label[0] != '\0'){
                  if(!new_search(label)) symtab = insert(new_operand, locctr);
            }
            if(lookup(op) != NULL) locctr += 3;
            else if(strcmp(op, a_word) == 0) locctr += 3;
            else if(strcmp(op, a_resw) == 0) locctr += 3 * atoi(operand);
            else if(strcmp(op, a_resb) == 0) locctr += atoi(operand);
            else if(strcmp(op, a_byte) == 0) locctr += operand_len();
      }
      prog_len = locctr - start_addr;
}

void print_symtab (symNODE * t) {
      if (t) {
            print_symtab( t->next );
            printf("[%s] = [%5X]\n", t->l, t->v);
      }
}

void init_obj_line () {
      int i;
      for (i=0; i<70; i++)
            obj_line[i] = ' ';
      obj_line[i] = '\0';
}

void wr_header () {
      fprintf(fobj, "H%-6s%06X%06X\n", prog_name, start_addr, prog_len);
}

void init_obj_code () {
      for (int i=0; i<6; i++) obj_code[i] = ' ';
      obj_code[6] = '\0';
}
```

```
void conv_byte ( int l, char *p, char *q ) {
	int i, j, k, max, c, m, n;
	if (p[0] == 'X') {
		max = 2 * l;
		for (i=2, j=0, k=0; k < max; i++, j++, k++)
			q[j] = p[i];
		q[j] = '\0';
	} else if (p[0] == 'C') {
		max = l;
		for (i=2, j=0, k=0; k < max; i++, k++) {
			c = (int)p[i];
			m = c / 16;
			q[j++] = hex_c[m];
			n = c % 16;
			q[j++] = hex_c[n];
		}
		q[j] = '\0';
	} else {
		printf("Error: wrong operand of BYTE!\n");
	}
}

void init_text () {
	init_obj_line();
	sprintf( obj_line, "T%6X    ", locctr );
	for (int i=1; i<7; i++)
		if (obj_line[i] == ' ') obj_line[i] = '0';
	textpos = 9;
}

void wr_text () {
	if(textpos > 9){
		int len = (textpos - 9) / 2;
		char hex_len[3];
		sprintf(hex_len, "%02X", len);
```

```
            obj_line[7] = hex_len[0];
            obj_line[8] = hex_len[1];
            obj_line[textpos] = '\0';
            fprintf(fobj, "%s\n", obj_line);
        }
}

void add_text ( int n, char *p ) {
        int const max = 69;
        int k = n * 2;
        int i;
        if ((textpos+k) > max) {
            wr_text();
            init_text();
        }
        for (i=0; i<k; i++) obj_line[textpos++] = p[i];
}

void wr_end () {
        fprintf(fobj, "E%06X\n", start_addr);
}

void pass2 () {
        char *opcode_hex;
        symNODE *sym_node;
        int op_addr;
        char temp_obj[60];
        locctr = start_addr;
        wr_header();
        init_text();
        while(readline()){
            if(strcmp(op, a_start) == 0) continue;
            if(strcmp(op, a_end) == 0) break;
            if((opcode_hex = lookup(op)) != NULL){
                if(operand[0] != '\0'){
```

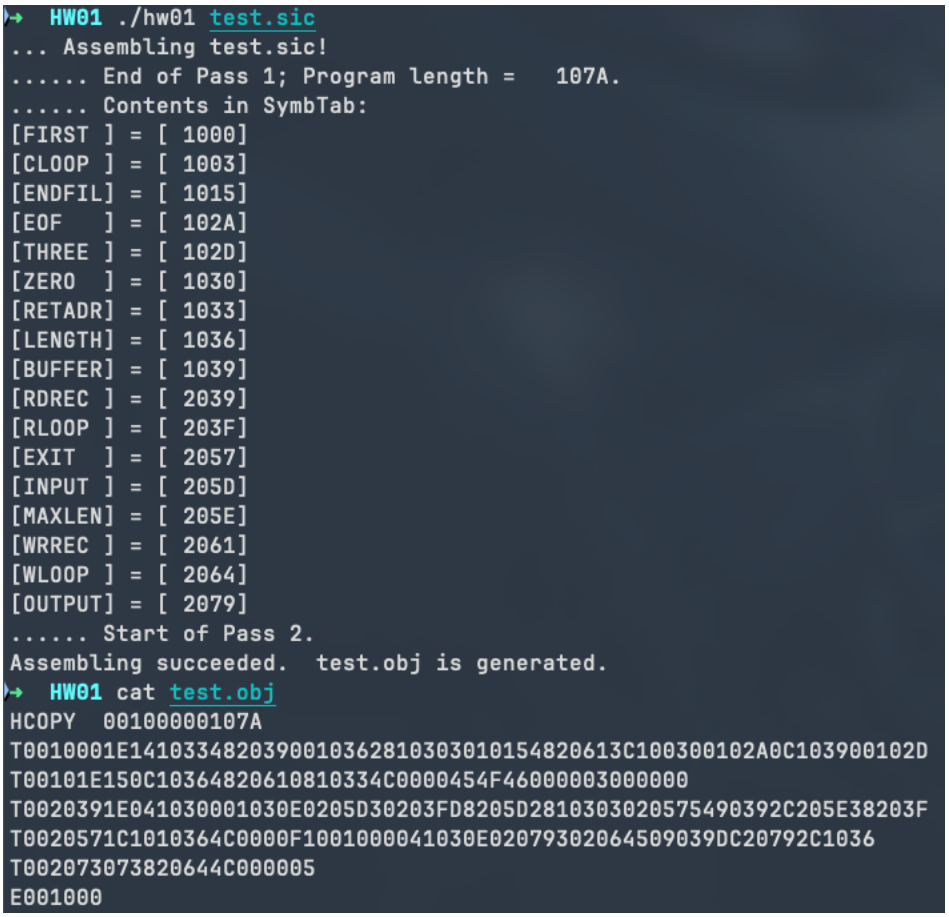
```
                sym_node = new_search(operand);
                if(sym_node != NULL) op_addr = sym_node->v;
                else op_addr = 0;
                if(indexed) op_addr += 0x8000;
            }
            else op_addr = 0;
            sprintf(obj_code, "%2s%04X", opcode_hex, op_addr);
            for(int i=0; i<6; i++)
                if(obj_code[i] == ' ') obj_code[i] = '0';
            add_text(3, obj_code);
            locctr += 3;
        }
        else if(strcmp(op, a_word) == 0){
            int val = atoi(operand);
            sprintf(obj_code, "%06X", val & 0xFFFFFF);
            add_text(3, obj_code);
            locctr += 3;
        }
        else if(strcmp(op, a_resw) == 0){
            wr_text();
            locctr += 3 * atoi(operand);
            init_text();
        }
        else if(strcmp(op, a_resb) == 0){
            wr_text();
            locctr += atoi(operand);
            init_text();
        }
        else if(strcmp(op, a_byte) == 0){
            int len = operand_len();
            conv_byte(len, operand, temp_obj);
            add_text(len, temp_obj);
            locctr += len;
        }
    }
```

```
	wr_text();
	wr_end();
}

int main(int argc, char*argv[]){
	int t = argc;
	char fname[20];
	int i = 0;
	if(t == 2) {
		f = fopen(argv[1], "r");
		if (f) {
			printf("... Assembling %s!\n", argv[1]);
			pass1();
			printf("..... End of Pass 1; Program length = %6X.\n", prog_len);
			printf("..... Contents in SymbTab:\n");
			print_symtab( symtab );
			fclose(f);
			strcpy(fname, argv[1]);
			for (i=0; (fname[i]!='.') && (fname[i]!='\0'); i++);
			fname[i++] = '.';
			fname[i++] = 'o';
			fname[i++] = 'b';
			fname[i++] = 'j';
			fname[i] = '\0';
			f = fopen(argv[1], "r");
			fobj = fopen(fname, "w");
			printf("..... Start of Pass 2.\n");
			pass2();
			printf("Assembling succeeded.    %s is generated.\n", fname);
			fclose(f);
			fclose(fobj);
		} else {
			printf("Assemble syntax: [assemble soure_file_name]\n");
		}
	} else {
```

```
            printf("Assemble syntax: [assemble soure_file_name]\n");
        }
}
```

D. Test run snapshots



E. Discussion

This assignment provided valuable hands-on experience with system-level programming in C. Through building the SIC assembler, I achieved the following learning outcomes:

1. Familiarity with Standard C I/O: I heavily utilized file I/O operations and formatted string outputs to handle the strict column-based formatting required by the object program.
2. String Parsing Challenges: Parsing the SIC assembly code was challenging due to the flexible use of spaces and the lack of strict delimiter rules. I had to manually write logic

to skip spaces and identify labels, opcodes, and operands. Handling the indexed addressing mode required careful string manipulation.

3. Memory Management: Implementing a linked list for the Symbol Table helped me practice dynamic memory allocation.
4. Format Constraints: Managing the Text record limit was a crucial part of the logic. I had to ensure the program correctly calculates the length of the current object codes and wraps to a new Text record (wr_text()) whenever it encounters a RESW/RESB directive or exceeds the line limit.

Overall, successfully generating an object file that exactly matches the course requirements gave me a deeper understanding of how source code is translated into machine-readable instructions.